

Rapport – Laboratoire 1

Nom : Simon Tremblay Code permanent : TRES01119908_
Cours : LOG430 – Architecture Logicielle

Session Été 2025

1. Informations générales

- URL du dépôt GitHub/GitLab : <https://github.com/EnzoTremblay/log430>
- Langage utilisé : python
- Base de données utilisée : SQL lite
- ORM utilisé : SQLAlchemy ORM

2. Auto-évaluation par section

2.1 Structuration du dépôt

Éléments	Oui	Non	Commentaire / Justification
Le fichier README.md est présent et complet	x		
Un fichier .gitignore pertinent est utilisé	x		
Le dépôt présente une structure claire et logique	x		

2.2 Application client

Éléments	Oui	Non	Commentaire / Justification
Elle permet de rechercher, ajouter, consulter, retourner des produits	x		
L'interface est claire et fonctionnelle	x		

2.3 Persistance et ORM

Éléments	Oui	Non	Commentaire / Justification
Un ORM est utilisé pour gérer la persistance	x		
L'application accède correctement à la base de données	x		
Les opérations de lecture et d'écriture sont persistées	x		
La base de données est cohérente après plusieurs opérations concurrentes	x		
L'utilisation des indexes dans la base de données est bien appliqué et justifié	x		

2.4 Justifications et documentation

Éléments	Oui	Non	Commentaire / Justification
Au moins deux décisions architecturales (ADR) sont rédigées	x		
Les choix technologiques sont documentés	x		
Les diagrammes UML sont présents (classes, séquence, déploiement)	x		
Les diagrammes sont faits avec un outil de diagramme comme code (diagram as code), tel que PlantUML ou Mermaid	x		
La documentation est écrite en Markdown	x		
La documentation suit un gabarit comme Arc42	x		
La documentation est organisée dans un répertoire dédié comme "docs" dans le répertoire du système	x		

2.6 Conteneurisation et exécution

Éléments	Oui	Non	Commentaire / Justification
L'application est conteneurisée avec un Dockerfile	x		
Client est conteneurisé	x		
La base de données instanciée manuellement	x		
La base de données instanciée avec système d'orchestration (ex. Docker compose)	x		
Les instructions d'exécution sont présentes dans le README	x		

2.7 Intégration continue (CI/CD)

Éléments	Oui	Non	Commentaire / Justification
Une pipeline CI/CD est configurée sur GitHub Actions ou GitLab CI	x		
Le lint est fonctionnelle	x		
L'étape test unitaire est fonctionnelle	x		
L'étape build Docker est fonctionnelle	x		
L'image est publiée sur Docker Hub	x		

3. Réflexion personnelle

- **Limites du style d'architecture client/serveur à 2 couches (2-tier)** :**
 - Difficulté à évoluer vers des architectures distribuées.
 - Risque de surcharge du serveur en cas de forte demande.
- **Limites du mécanisme de base de données utilisée** :**
 - SQLite est limité en termes de concurrence et de scalabilité.
 - Tactiques pour améliorer la capacité de réponse :
 - Utilisation de caches.
 - Optimisation des requêtes.
 - Migration vers une base de données plus robuste comme PostgreSQL.
- **Efforts techniques et risques liés à la migration vers une base NoSQL** :**
 - Efforts :
 - Réécriture des requêtes.
 - Adaptation des modèles de données.
 - Risques :
 - Perte de cohérence transactionnelle.
 - Complexité accrue pour les relations complexes.
- **Éléments posant le plus de difficulté** :**
 - Configuration de la pipeline CI/CD.
 - Gestion des secrets dans le dépôt.
- **Améliorations pour les prochains laboratoires** :**
 - Automatisation accrue des tests.
 - Documentation plus détaillée des choix technologiques.